The Journal of Open Source Software

AnyPINN: A Modular, Training-Agnostic Python Library for Physics-Informed Neural Networks

Giacomo Guidotto [1], **Caterina Millevoi** [2], **and Damiano Pasetto** [1]

1 Ca' Foscari University of Venice, Venice, IT **2** Università di Padova, Padova, IT

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 05 May 2026
Published: unpublished

Summary

AnyPINN is a PyTorch-native library for Physics-Informed Neural Networks (PINNs) as composable `nn.Module` objects. It supports both ODE and PDE problems, forward and inverse alike, with particular depth in the inverse setting where parameters must be recovered from partial observations. The library is organised into four layers (`core`, `problems`, `lightning`, `catalog`) with a strict one-way dependency direction, so users can engage with exactly the layer that matches their expertise, from running a pre-built example through the bootstrap CLI to defining custom physics and training loops in plain PyTorch. All hyperparameters are plain Python dataclasses: serializable, diffable, and programmatically constructible, which makes systematic ablation studies a matter of iterating over configuration objects rather than editing files. Training is not owned by the library: `Problem.training_loss()` returns a scalar tensor compatible with any optimizer or training infrastructure, including plain PyTorch and PyTorch Lightning.

Statement of Need

Physics-Informed Neural Networks encode differential equation residuals as loss terms, enabling mesh-free, automatic-differentiation-based solutions to both forward and inverse problems (Cuomo et al., 2022; Karniadakis et al., 2021; Lagaris et al., 1998; Raissi et al., 2019).

Despite a growing ecosystem of PINN libraries (F. Chen et al., 2021; Cuomo et al., 2022; Lu et al., 2021), two pain points persist. First, every major library couples physics definition to its own training loop, preventing researchers from using the full PyTorch potential. Second, inverse problems remain second-class: unknown parameters are configuration flags rather than typed objects, and ground-truth tracking during training requires user-written callbacks.

AnyPINN addresses both gaps. The core abstraction, `Problem`, is a standard `nn.Module` whose `training_loss()` returns a scalar tensor, compatible with any optimizer, scheduler, or distributed strategy without library-specific wrappers. For inverse problems specifically, `Parameter` is a first-class type with the same `forward(x)` interface whether it wraps a fixed scalar or a time-varying MLP; switching from fixed to learnable requires changing one configuration line. `ValidationRegistry` binds ground-truth references to parameter names and logs MSE automatically at every training step. `ArgsRegistry` unifies fixed and learnable arguments so that the differential equation callable cannot distinguish between the two, enabling transparent composability.

The target audience ranges from researchers defining novel physics to engineers running parameter sweeps on pre-built models. The bootstrap CLI (`anypinn create`) scaffolds a complete project from a single interactive command, making the library accessible to users who have never written a PINN before. Researchers who need full control can bypass the CLI and the Lightning layer entirely, working directly with `anypinn.core` and plain PyTorch.

State of the Field

The PINN software ecosystem spans more than ten libraries across two major frameworks (Cuomo et al., 2022; Karniadakis et al., 2021). Table 1 compares the libraries most relevant to AnyPINN's scope across several dimensions.

Table 1: Comparison of AnyPINN with existing PINN libraries. "Inverse (first-class)" means the library provides typed parameter objects, unified fixed/learnable interfaces, and built-in ground-truth tracking, not merely the ability to mark a variable as trainable.

Library	Framework	PDE support	Inverse (first-class)?	Scaffold CLI?	Engine-agnostic?
NeuroDiffEq (F. Chen et al., 2021)	PyTorch	2D	No	No	No
IDRLNet	PyTorch	Yes	No	No	No
NVIDIA Modulus (NVIDIA Corporation, 2023)	PyTorch	Yes (3D)	No	No	No
PINA (Coscia et al., 2024)	PyTorch	Yes	No	No	No
DeepXDE (Lu et al., 2021)	TF / JAX / PyTorch	Yes	No	No	No
TensorDiffEq	TensorFlow	Yes	No	No	No
SciANN	TensorFlow	Yes	No	No	No
PyDEns	TensorFlow	Limited	No	No	No
Elvet	TensorFlow	Limited	No	No	No
NVIDIA SimNet	TensorFlow	Yes (3D)	No	No	No
AnyPINN	**PyTorch**	**Yes**	**Yes**	**Yes**	**Yes**

Several libraries in this comparison are no longer actively maintained: SciANN is explicitly deprecated, TensorDiffEq has seen no activity since 2022, NVIDIA SimNet has been superseded by Modulus (now PhysicsNeMo), and both PyDEns and IDRLNet have had no meaningful development since 2023. This further narrows the set of viable options for researchers starting new PINN projects.

Three structural patterns in the existing ecosystem motivate AnyPINN as a separate library rather than a contribution to an existing one.

Framework distribution. More than half of the libraries in the comparison set are TensorFlow-based. PyTorch (Paszke et al., 2019) has become the dominant framework for ML research, and its downstream ecosystem (PyTorch Lightning (Falcon & The PyTorch Lightning team, 2019), Hugging Face, `torch.compile`, Triton) has grown substantially. DeepXDE has added PyTorch and JAX backends, but was originally designed around TensorFlow idioms. A researcher already working in PyTorch adopting a TensorFlow-native PINN library inherits framework friction unrelated to the science.

Training engine coupling. Every major competitor couples physics definition to its own training loop: NeuroDiffEq exposes a `solve_ivp`-style API; DeepXDE requires calling `Model.train()`; IDRLNet uses a proprietary computational graph; NVIDIA Modulus ships its own distributed `Trainer`; PINA wraps Lightning with an additional abstraction layer. When the training ecosystem evolves with new optimizers, new precision formats, new distributed strategies, users of these libraries must wait for library-level support or write wrappers around internal abstractions. AnyPINN's `Problem` is a plain `nn.Module`; the library never owns the training loop.

Inverse problem primitives. All listed libraries can, in principle, solve inverse problems by marking variables as trainable. However, none ship `Parameter` as a typed object with a unified interface, a `ValidationRegistry` for ground-truth tracking, or an `ArgsRegistry` that makes fixed and learnable arguments indistinguishable to the equation callable. In libraries such as DeepXDE and NeuroDiffEq, the distinction between fixed and learnable parameters is visible at the problem-definition level, and ground-truth comparison requires user-written callbacks.

Software Design

AnyPINN is split into four layers with strict one-way dependency (outer depends on inner, never the reverse).

`anypinn.core`: pure PyTorch. The mathematical foundation defines what a PINN problem *is* without any opinion about training. `Problem` (an `nn.Module`) aggregates `Constraint` instances, fields, and parameters and exposes `training_loss(batch, log)` returning a scalar tensor. `Field` is an MLP mapping input coordinates to state variables. `Parameter` inherits from `Argument` and exposes `forward(x)` whether it wraps a fixed scalar or a learnable MLP, making it transparent to the equation callable. `ArgsRegistry` is a typed dictionary of `Argument` instances; `InferredContext` extracts domain bounds from data and injects them into constraints automatically. Because `Problem` is a standard `nn.Module`, it participates transparently in the full PyTorch ecosystem: mixed precision, `torch.compile`, distributed data parallel, and gradient checkpointing require no library-level changes.

`anypinn.problems`: ODE and PDE building blocks. For ODE inverse problems, `ResidualsConstraint` enforces $\|\dot{y} - f(t, y)\|^2$ via autograd with support for arbitrary-order ODEs through chained derivatives. `ICConstraint` enforces $\|y(t_0) - y_0\|^2$ and natively handles higher-order initial conditions. `DataConstraint` enforces $\|y - y_{\text{obs}}\|^2$ against observed data. `ODEInverseProblem` composes all three with configurable loss weights and a selectable loss criterion (MSE, Huber, or L1). For PDE problems, `DirichletBCConstraint` and `NeumannBCConstraint` enforce boundary conditions on sampled boundary points, and `PDEResidualConstraint` enforces an arbitrary residual function over interior collocation points with field-subset scoping for coupled systems. The composable differential operators in `anypinn.lib.diff` (`grad`, `partial`, `mixed_partial`, `laplacian`, `divergence` and `hessian`) are available for use inside any constraint definition.

`anypinn.lightning`: optional training wrapper. A thin PyTorch Lightning (Falcon & The PyTorch Lightning team, 2019) layer for users who want batteries-included training. `PINNModule` wraps any `Problem` as a `LightningModule`; `PINNDataModule` manages data loading and collocation sampling. Five collocation strategies are available via a configuration string: `"uniform"`, `"random"`, `"latin_hypercube"`, `"log_uniform_1d"`, and `"adaptive"` (residual-weighted, with configurable exploration ratio). Callbacks provide SMMA-based early stopping, data scaling, and prediction writing. The entire Lightning layer is optional: `anypinn.core` has no Lightning dependency.

`anypinn.catalog`: drop-in problem modules. Pre-built ODE functions and data modules for SIR, SEIR, damped oscillator, and Lotka-Volterra systems. Catalog entries are runnable out of the box and serve as concrete starting points for the bootstrap CLI. The catalog is designed to be extended with additional dynamical systems as the library grows.

Configuration as data. All hyperparameters such as MLP architecture, optimizer settings, collocation strategy, loss weights, are expressed as keyword-only Python dataclasses. These objects are plain data: they can be serialized, loaded from configuration files, diffed across runs, and constructed programmatically in loops. This makes systematic ablation studies i.e., sweeping over learning rates, collocation counts, or loss weights, a matter of iterating over configuration objects in a script rather than editing files by hand.

Input encodings. `FourierEncoding` and `RandomFourierFeatures` are available as `nn.Module`

input encodings, lifting low-frequency MLPs to high-frequency solutions without changes to the training loop.

The four-step workflow for a custom inverse problem is:

```python
# 1. Define the ODE
def my_ode(x: Tensor, y: Tensor, args: ArgsRegistry) -> Tensor:
    k = args["k"](x)   # same interface for fixed or learnable k
    return -k * y

# 2. Configure hyperparameters
@dataclass(frozen=True, kw_only=True)
class MyHP(ODEHyperparameters):
    pde_weight: float = 1.0
    ic_weight: float = 10.0
    data_weight: float = 5.0

# 3. Build the problem
props = ODEProperties(ode=my_ode, args={"k": param}, y0=y0)
problem = ODEInverseProblem(ode_props=props, fields={"u": field},
                            params={"k": param}, hp=hp)

# 4. Train with Lightning or plain PyTorch
optimizer = torch.optim.Adam(problem.parameters(), lr=1e-3)
for batch in dataloader:
    optimizer.zero_grad()
    loss = problem.training_loss(batch, log=my_log_fn)
    loss.backward()
    optimizer.step()
```

The bootstrap CLI (`anypinn create`) generates a complete, immediately runnable four-file project (`ode.py`, `config.py`, `train.py`, `pyproject.toml`) from a single interactive command, lowering the barrier for users unfamiliar with PINN library APIs.

Research Impact

AnyPINN ships six working examples spanning epidemiology (SIR with real Italian COVID-19 data, SEIR, reduced-SIR with hospitalization dynamics), mechanics (damped oscillator: recovering damping ratio and natural frequency), and ecology (Lotka-Volterra predator-prey dynamics), plus a minimal exponential decay script (~80 lines, core only, no Lightning). The SIR example reproduces the PINN methodology of Millevoi et al. (2024), serving as a validation benchmark: AnyPINN recovers comparable transmission rate dynamics and state variable trajectories on the same Italian COVID-19 dataset. Each example demonstrates a different dynamical system, scientific domain, and data type, with generated result plots and CSV exports. Neural ODEs (R. T. Q. Chen et al., 2018) offer a complementary approach that replaces the ODE right-hand side with a neural network entirely; AnyPINN instead keeps the equation explicit, which makes the same `Problem` and `Constraint` abstractions applicable to both forward solutions and parameter recovery. The PDE constraint layer (`DirichletBCConstraint`, `NeumannBCConstraint`, `PDEResidualConstraint`) and the composable differential operators in `anypinn.lib.diff` extend the library's applicability beyond ODE systems, providing a foundation for spatiotemporal forward and inverse problems alike.

AI Usage Disclosure

Claude (Anthropic) was used during the development of the AnyPINN codebase to assist with code audit, bug identification, and targeted fixes. All architectural decisions, design choices, and major implementations were authored by the human developer. Claude was not used to generate the core library code or its scientific content.

References

Chen, F., Sondak, D., Protopapas, P., Mattheakis, M., Liu, S., Agarwal, D., & Di Giovanni, M. (2021). NeuroDiffEq: A Python package for solving differential equations with neural networks. *Journal of Open Source Software*, *6*(61), 3008. https://doi.org/10.21105/joss.03008

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, *31*. https://proceedings.neurips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html

Coscia, D., Meneghetti, L., Demo, N., Stabile, G., & Rozza, G. (2024). *PINA: Physics-Informed Neural networks for Advanced scientific computing*. GitHub repository. https://github.com/mathLab/PINA

Cuomo, S., Di Cola, V. S., Giampaolo, F., Rozza, G., Raissi, M., & Piccialli, F. (2022). Scientific machine learning through physics-informed neural networks: Where we are and what's next. *Journal of Scientific Computing*, *92*(3), 88. https://doi.org/10.1007/s10915-022-01939-z

Falcon, W., & The PyTorch Lightning team. (2019). *PyTorch Lightning*. Zenodo. https://doi.org/10.5281/zenodo.3828935

Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, *3*(6), 422–440. https://doi.org/10.1038/s42254-021-00314-5

Lagaris, I. E., Likas, A., & Fotiadis, D. I. (1998). Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, *9*(5), 987–1000. https://doi.org/10.1109/72.712178

Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, *63*(1), 208–228. https://doi.org/10.1137/19M1274067

Millevoi, C., Pasetto, D., & Ferronato, M. (2024). A physics-informed neural network approach for compartmental epidemiological models. *PLOS Computational Biology*, *20*(9), 1–29. https://doi.org/10.1371/journal.pcbi.1012387

NVIDIA Corporation. (2023). *NVIDIA Modulus: A physics-ML platform*. Online documentation. https://docs.nvidia.com/deeplearning/modulus/index.html

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, *378*, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045